> React components can be either functions that return a JSX element or a class that contains a `render` method that returns a JSX element. Until recently, only class components could maintain internal state, but that changed in React 16.8 with the introduction of *hooks*. It seems to be the clear direction of the React team that functional components are the future, so that's what we'll deal with here, but if you look at a lot of older React code, you'll see a lot of class components.

Looking at our `Seat` function, we see that it takes one argument called `props` and does nothing but return an element that uses JSX to describe an HTML table cell. The return value encloses the JSX in parentheses, which is a common idiom for returning multi-line JSX elements, allowing you to start the JSX element on the line after the `return` statement and preserve logical indenting.

The returned value starts with `<td>`. When a JSX file sees an angle bracket, it looks to convert the expression contained by that angle bracket element to a React element using an internal React function. (JSX is mostly a shortcut for calling React's tag creation functions all over the place, which would be horribly verbose.) If the initial text inside the angle brackets, in this case `td`, starts with a lowercase letter, React assumes it is an HTML element. If it starts with an uppercase letter, React assumes it's another React component.

Using JSX, we can include HTML attributes in our React components almost identically to how we would in normal HTML markup. You can see one of the main exceptions in this code. Since `class` is a reserved word in JavaScript, DOM classes are added to elements using the attribute name `className`.

Inside our span element we have `{props.seatNumber}`. This syntax uses two parts of